

Lab Report: Magnetic Field of a Solenoid

Chhaya Ramnani (UID: u8336188)

Lab partners: Tsz Wing Candice Ng, Chinatsu Komada

2026-08-25

Imports and Functions

Same deal as last time- errors on both axes, so it's `scipy.odr` again, but not a linear fit

Starting with imports:

```
from dataclasses import dataclass
import matplotlib.pyplot as plt
import numpy as np
from scipy import odr, constants, stats
from uncertainties import ufloat
```

Defining the constants for the solenoid setup. N is fixed (given: 700 turns) and μ_0 we're pulling straight from `scipy.constants` instead of typing out (because whimsy and why not).

Then measured the length of the solenoid. For the radius, we measured both the outer and inner radius of the coil form and took the average of the two as our effective R . We had no current pass through the probe and measured the atmospherical V_0 .

For all of our length measurements, we measure in increments of 0.5cm, so our uncertainty is half of that, 0.25cm. From the lab equipment notes our current has a 1% error in our ammeter and our voltage_0 we have a 1.2% error plus the smallest digit.

```
N = 700
mu_0 = constants.mu_0 # (Wb / A m)

r_outer = ufloat(5.7, 0.25) / 100
r_inner = ufloat(3.8, 0.25) / 100

R_solenoid = (r_outer + r_inner) / 2 # m
L_solenoid = ufloat(14.9, 0.25) / 100 # m
I_solenoid = ufloat(2.32, 2.32 * 0.01) # A
```

```
V_0 = ufloat(2.52, 2.52*0.01) # V
S_sensor = ufloat(3.13, 0.09) * 0.001 / 0.0001 # sensitivity
```

`solenoid_model` is just equation 1 typed out directly with `scale` and `offset`. `scale` lets the fit stretch the whole predicted curve up or down (should land near 1 if the raw theory already matches), and `offset` lets it slide up or down by a constant (mostly there to catch a slightly-off V_0 measurement).

```
@dataclass
class odr_fit_result:
    parameters: np.ndarray
    parameter_errors: np.ndarray
    r_squared: float
    rmse: float
    reduced_chi_squared: float
    converged: bool
    x_nom: np.ndarray
    y_nom: np.ndarray
    x_err: np.ndarray
    y_err: np.ndarray

def solenoid_model(p, d):
    scale, offset = p
    L = L_solenoid.n
    R = R_solenoid.n
    I = I_solenoid.n

    term1 = (d + L / 2) / np.sqrt((d + L / 2) ** 2 + R ** 2)
    term2 = (d - L / 2) / np.sqrt((d - L / 2) ** 2 + R ** 2)
    B_z = (mu_0 / (4 * np.pi)) * (2 * np.pi * N * I / L) * (term1 - term2)

    return scale * B_z + offset
```

`odr_fit` extracts nominal values and uncertainties out of our ufloats, runs the fit, and this time also computes a proper reduced chi-squared straight from `odr_result.res_var`

```
def odr_fit(x: list[ufloat], y: list[ufloat]) -> odr_fit_result:
    x_nom = np.array([v.n for v in x])
    x_err = np.array([v.s for v in x])
    y_nom = np.array([v.n for v in y])
    y_err = np.array([v.s for v in y])

    data = odr.RealData(x_nom, y_nom, sx=x_err, sy=y_err)
```

```

model = odr.Model(solenoid_model)
beta0 = [1.0, 0.0]

odr_result = odr.ODR(data, model, beta0=beta0).run()

parameters = odr_result.beta
parameter_errors = odr_result.sd_beta
converged = odr_result.info in (1, 2, 3)
if not converged:
    print(f"did not converge (info={odr_result.info})")

y_pred = solenoid_model(parameters, x_nom)
residuals = y_nom - y_pred
ss_res = np.sum(residuals**2)
ss_tot = np.sum((y_nom - np.mean(y_nom)) ** 2)
r_squared = 1 - ss_res / ss_tot if ss_tot != 0 else np.nan
rmse = np.sqrt(np.mean(residuals**2))
reduced_chi_squared = odr_result.res_var

return odr_fit_result(
parameters=parameters,
parameter_errors=parameter_errors,
r_squared=r_squared,
rmse=rmse,
reduced_chi_squared=reduced_chi_squared,
converged=converged,
x_nom=x_nom,
y_nom=y_nom,
x_err=x_err,
y_err=y_err,
)

```

two plotting functions this time !! yay. `plot_field` fits our data against the theoretical curve, and `plot_residuals` shows just the leftover gaps between data and fit.

```

def plot_field(x, y, sx, sy, fit_result, xlabel, ylabel, title, filename):
    x_fit = np.linspace(min(x), max(x), 200)
    y_fit = solenoid_model(fit_result.parameters, x_fit)

    plt.figure(figsize=(7, 5))
    plt.errorbar(x, y, xerr=sx, yerr=sy, fmt="o", capsize=3, label="Experimental
Data (Hall Sensor)")
    plt.plot(x_fit, y_fit, "k--",

```

```

label=f"Fit ( $\chi^2 = \{fit\_result.reduced\_chi\_squared:.2f\}$ )"
plt.xlabel(xlabel)
plt.ylabel(ylabel)
plt.title(title)
plt.grid(True, alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(filename, dpi=300)
plt.show()

def plot_residuals(x, y, sy, fit_result, xlabel, filename):
    residuals = y - solenoid_model(fit_result.parameters, x)

    plt.figure(figsize=(7, 4))
    plt.errorbar(x, residuals, yerr=sy, fmt="o", capsize=3, color="crimson")
    plt.axhline(0, color="black", linestyle="--")
    plt.xlabel(xlabel)
    plt.ylabel("Residual (T)")
    plt.title("Residuals")
    plt.grid(True, alpha=0.6)
    plt.tight_layout()
    plt.savefig(filename, dpi=300)
    plt.show()

```

along with the sigma test, we're using `scipy.stats.chi2` to get an actual p-value out of it rather than eyeballing a “is it between 0.5 and 1.5” rule of thumb. the p-value is the probability of seeing a mismatch this big (or bigger) between data and theory.

```

def chi_squared_test(chi2_stat, dof):
    p_value = stats.chi2.sf(chi2_stat, dof) # sf = 1 - cdf, "survival function"
    print(f"chi-squared: {chi2_stat:.3f} (dof = {dof})")
    print(f"reduced chi-squared: {chi2_stat/dof:.3f}")
    print(f"p-value: {p_value:.4f}")
    if p_value > 0.05:
        print("results AGREE with the model (p > 0.05)")
    else:
        print("results DISAGREE with the model (p <= 0.05)")
    return p_value

def sigma_test(k1, sk1, k2, sk2):
    sigma_diff = abs(k1 - k2) / np.sqrt(sk1**2 + sk2**2)

    print("\nSIGMA TEST")

```

```

print(f"measured scale factor    : {k1:.3f} +/- {sk1:.3f}")
print(f"theoretical scale factor : {k2:.3f} +/- {sk2:.3f}")
print(f"discrepancy            : {sigma_diff:.2f} sigma")

if sigma_diff <= 3.0:
    print("results AGREE within experimental uncertainty")
else:
    print("results DISAGREE")

return sigma_diff

```

Pre-lab Question:

To predict how the maximum induced emf depends on the number of turns (N), the radius of the coil (R) and the speed of the magnet (s).

Number of turns: Faraday's law states that $\varepsilon = -N \frac{d\Phi_B}{dt}$. Because each loop of wire acts in series, the total voltage induced adds up across all N turns. This means the induced emf scales in a direct, straight-line linear relationship with the number of turns—if you double N , the peak emf doubles.

Radius of the coil: Magnetic flux depends on the area of the coil ($\Phi_B = \int B_{\perp} dA$). While a larger radius means a bigger area to capture more flux, a magnet's magnetic field gets significantly weaker the further you get from its center. Because you start capturing much weaker peripheral field lines at larger radii, the exact functional form depends on how the magnetic field varies across that larger area

Speed of the magnet: Using the chain rule, the rate of change of flux can be split into how the flux changes with position ($d\Phi_B/dz$) multiplied by how fast position changes with time (dz/dt). Since dz/dt is the speed of the magnet, the induced emf should be directly proportional to s , meaning $\varepsilon \propto s$. If the magnet falls twice as fast, the flux changes twice as fast, doubling the induced emf.

Aim:

Theory:

Procedure:

Results

Observations

pretty happy with how this one turned out. The shape is exactly what equation 1 says it should be- flat through the middle, then it rolls off nice and smooth once the probe passes about $d = 6.5$ cm and starts leaving the coil. $R^2 = 0.9505$, **p-value of 0.5728**, **reduced $\chi^2_v = 0.921$** - basically bang on 1.

but our fitted **scale factor came out at 1.2421 ± 0.0544** , and that's 4.5σ away from the 1.0 we'd want if the raw theory already matched us with zero adjustment. So even though the shape is great, that's a systematic offset, and a pretty resolved one given how tight that uncertainty is.

Where's it coming from? We used the average of the inner and outer coil radius as our effective R , but the actual windings could easily sit closer to one edge than dead centre, and since R shows up nonlinearly in Equation 1, getting it even a little wrong tilts the whole predicted curve. second, we just took the manufacturer's 3.13 ± 0.09 mV/G sensitivity at face value- if our specific probe runs a bit off spec, that'd mess it up a bit.

Limitations & possible sources of discrepancy:

- Radius is a guess, not a measurement of the windings themselves. Averaging inner and outer radius is a reasonable stand-in, but it's not the same as actually knowing where the current loops sit.
- Data isn't symmetric about the centre. We only really swept in detail from -5 cm to $+18$ cm, with denser sampling on the positive side past the coil's edge- so our fit is leaning more heavily on one side of the solenoid than the other, which could bias the fitted scale factor if there's any asymmetry in the coil itself or uneven winding, slightly tilted probe, etc etc.
- Only one current setting tested. We fixed $I = 2.32$ A the whole time. If our scale-factor mismatch is actually a current-reading issue, we'd have no way to tell from this data alone- repeating at a couple of different currents and checking if the same scale factor shows up each time would help isolate it.
- The sensor only picks up the field component perpendicular to its flat face. if the probe wasn't held perfectly parallel to the solenoid's axis the whole time, we'd be undercounting the true axial field, which would push the scale factor. not the main culprit, but a possible source of error